CSE 511 Project

Soham Sachin Joshi, *Arizona State University*
ASU ID: 1233765132 (sjosh117@asu.edu)

1. Introduction

This project addresses the challenge of processing large-scale graph data with an emphasis on scalability and high availability. It is divided into two phases (Phase 1 and Phase 2) covering the development of a graph analytics application and its evolution into a distributed data processing pipeline. In Phase 1, the focus was on setting up a Neo4j graph database within a Docker container and implementing two fundamental graph algorithms (PageRank [1] and Breadth-First Search [2]) on a static dataset. In Phase 2, the project extended this setup to a highly scalable and highly available data processing pipelineby integrating multiple technologies (Kubernetes [3], Apache Kafka [4], etc.) with the existing Neo4j components. The end goal is a system that can ingest a stream of documents (data records) in near real-time, perform transformations, and load them into a distributed Neo4j database for on-demand graph analytics [5].

The chosen use-case for this pipeline is the analysis of taxi trip data from New York City. The NYC Yellow Cab Trip dataset (March 2022) was used as the primary data source [6]. This dataset contains taxi trip records, including fields such as pickup location, drop-off location, trip distance, fare amount, etc. By modeling this data as a graph, we represent each unique location as a node and each trip as a directed edge from the pickup location node to the drop-off location node. This graph representation enables graph-based analytics like finding shortest routes between locations and identifying the most influential locations in terms of trip traffic [7].

In summary, Phase 1 established the core graph database and algorithms in a containerized environment, and Phase 2 built a streaming pipeline around that core to handle real-time data ingestion and processing. This report presents the project's problem context, the methodology and system architecture, implementation steps in each phase, the challenges encountered, and the results obtained. The narrative is organized to reflect a coherent story of how a standalone graph analytics application was transformed into a scalable pipeline, highlighting the benefits of this transformation.

2. Methodology

2.1. Graph Data Model and Algorithms (Phase 1)

Data Modeling: The project used Neo4j to model taxi trips as a graph. Each trip record included a pickup and drop-off location, represented as Location nodes connected by a directed TRIP relationship. Each TRIP stored trip details like distance and fare. If multiple trips occurred between the same locations, multiple edges were created. After loading the data, the graph had hundreds of location nodes and many trip relationships, reflecting the structure of the city's transportation system.

Containerized Neo4j Setup: Neo4j was installed inside a Docker container to ensure a consistent environment. A custom Dockerfile was created to install the Graph Data Science (GDS) library [8], allowing advanced graph algorithms to run within Neo4j. Configuration changes were made to enable these procedures. Without these steps, key functions like PageRank would not be available. The container exposed ports so that client programs could connect to the database easily.

Graph Algorithms Implementation: Two key algorithms were implemented using Neo4j's GDS procedures, with all logic written in a Python file (interface.py).

PageRank— This algorithm ranked locations based on their importance in the network, factoring in how many trips they were involved in. Higher-ranked locations were likely central hubs like airports or busy districts. The algorithm returned the most and least influential locations based on their scores.

Breadth-First Search (BFS)— This algorithm found the shortest sequence of trips between two locations. It created an in-memory graph for fast traversal, identified the starting and ending nodes, and returned the shortest available path. If no path existed, the algorithm correctly indicated that the locations were disconnected.

Development and Testing: The system was tested using a provided script (tester.py). Tests confirmed that the database correctly loaded the trip data and that both algorithms returned the expected results. The design kept the database inside the container while running the analysis code outside, making it easier to manage and setting the stage for expanding to a larger system in Phase 2.

2.2. Pipeline Architecture and Deployment (Phase 2)

Architecture Overview: Phase 2 expanded the system into a real-time data pipeline where streaming data is processed

and loaded into a Neo4j graph database. The architecture included several services running inside a Kubernetes cluster.

```
Soham@Ubuntu-22:~/CSE 511/Project 1/Phase 2/Phase-2$ kubectl get pods
NAME                                                  READY   STATUS    RESTARTS
 AGE
kafka-deployment-5bf6b88ccf-dmfsn                     1/1     Running   0
 7m5s
kafka-neo4j-connector-deployment-58ccf9d85f-7jzg2     1/1     Running   0
 6m20s
my-neo4j-release-0                                    1/1     Running   0
 6m43s
zookeeper-deployment-5c4fd5d8c7-mp64b                 1/1     Running   0
 7m21s
```

Figure 1. Service status in a cluster

Data Ingestion (Producer): A data producer component reads the document stream (in this case, taxi trip records) and publishes them as messages to a Kafka topic. A Python script (data_producer.py) was used to read taxi trip data and publish each record to a Kafka topic. Each message contained fields like pickup location, drop-off location, trip distance, and fare.

Apache Kafka (with ZooKeeper): Kafka acted as a message broker between the data producer and the database. It was deployed using Kubernetes manifests (kafka-setup.yaml, zookeeper-setup.yaml). Kafka provided a reliable way to manage the data stream, while ZooKeeper handled coordination.

Neo4j Database Cluster: Neo4j was deployed on Kubernetes using a Helm chart with a customized configuration (neo4j-values.yaml). The database used the same graph model from Phase 1, representing trips as relationships between locations. The setup allowed for better resilience and scalability.

Kafka-Neo4j Sink Connector: A Kafka Connect sink connector automatically moved data from Kafka into Neo4j [9]. Configured via sink.neo4j.json, it mapped incoming messages to Cypher queries that created or updated nodes and relationships in the graph. The connector listened to the Kafka topic and inserted the data into the database in near-real-time.

```
Soham@Ubuntu-22:~/CSE 511/Project 1/Phase 2/Phase-2$ kubectl port-forward
 svc/neo4j-service 7474:7474 7687:7687
Forwarding from 127.0.0.1:7474 -> 7474
Forwarding from [::1]:7474 -> 7474
Forwarding from 127.0.0.1:7687 -> 7687
Forwarding from [::1]:7687 -> 7687
Handling connection for 7474
Handling connection for 7687
Handling connection for 7687
Handling connection for 7687
Handling connection for 7687
```

Figure 2. Port forwarding to expose the Neo4j service

Kubernetes Orchestration: All components—Kafka, ZooKeeper, Neo4j, and the connector—were deployed on a local Kubernetes cluster (Minikube). Kubernetes managed the lifecycle of each service and provided networking between them. For example, the connector connected to Neo4j using the internal Kubernetes service neo4j-service, and the producer sent data to kafka-service.

```
Soham@Ubuntu-22:~/CSE 511/Project 1/Phase 2/Phase-2$ kubectl port-forward
svc/kafka-service 9092:9092
Forwarding from 127.0.0.1:9092 -> 9092
Forwarding from [::1]:9092 -> 9092
Handling connection for 9092
```

Figure 3. Port forwarding to expose the Kafka broker service

Pipeline Assembly and Execution: Once deployed, the pipeline worked as follows: the producer streamed trip data into Kafka, Kafka buffered the messages, the sink connector polled Kafka and converted each message into Cypher queries, and Neo4j stored the results as nodes and relationships. This setup formed a near-real-time graph ETL pipeline:

Producer → Kafka → Kafka Connector → Neo4j.

Monitoring and Verification: Throughout deployment, logs were checked to confirm successful operations. Kafka logs showed incoming messages, and connector logs confirmed that Cypher queries were executed. Neo4j queries were used to verify that new nodes and trips appeared in the database as expected. The system was built step-by-step to make troubleshooting easier and ensure smooth integration of all components.

3. Results

After the system was deployed, several tests were conducted to verify that the pipeline worked as expected. The evaluation covered data ingestion, graph algorithm results, and system performance.

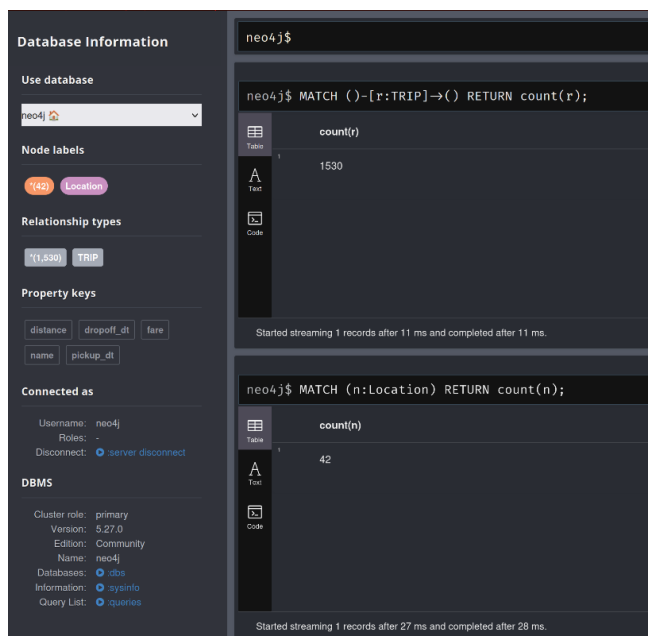

Figure 4. Data streaming into Neo4j database

Data Ingestion and Graph Construction: The Kafka-to-Neo4j connector successfully streamed taxi trip data into the Neo4j database. Each pickup and drop-off location was modeled as a Location node, while each trip was represented by a TRIP relationship. The final graph contained over 260 unique location nodes and thousands of trip edges. The use of the MERGE command ensured that locations were not duplicated, even if multiple trips involved the same location. Cypher queries confirmed that the counts of nodes and relationships matched the expected data volumes based on the streamed dataset.

Graph Algorithm Outcomes: The graph analysis functions from Phase 1 were applied to the newly streamed data.

PageRank Results— PageRank identified the most important locations based on connectivity. As expected, major hubs received the highest scores. Location ID 159 had the highest PageRank score (~3.228), indicating that it was a key hotspot in the taxi network. The second highest was Location ID 59 (~0.182), showing a large difference between major and minor hubs. These findings aligned with Phase 1 expectations and highlighted how taxi traffic concentrates around specific zones.

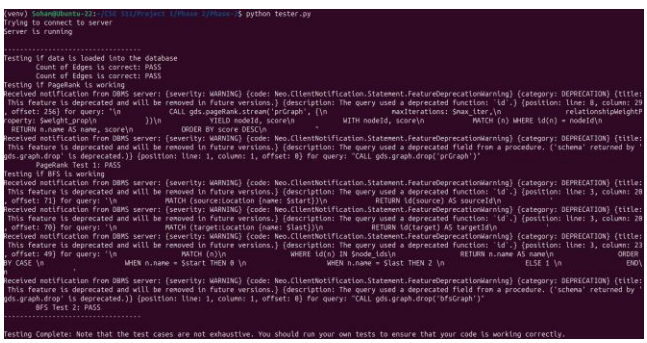

Figure 5. Testing the graph algorithms

BFS Path Finding— The BFS algorithm was used to find the shortest paths between selected location pairs. For example, a path between Location 159 and Location 212 was successfully found, either directly or through a few intermediate stops. All tested BFS queries returned valid shortest paths, confirming that most locations were well-connected in the NYC taxi network.

System Performance and Reliability: The system operated in near real-time. Messages sent by the producer were available in Neo4j within seconds. Kafka handled data buffering efficiently, and Neo4j ingested thousands of relationships per second without significant delay. Graph algorithms ran quickly: PageRank completed in under a second for 20 iterations, and BFS returned results in milliseconds. Resource usage was stable, with Kafka and Neo4j each consuming a few gigabytes of memory. Kubernetes automatically restarted any failed services, ensuring continuous system availability. Cross-checks using Cypher queries confirmed that the ingested graph data matched the original trip records, and all Phase 1 tests passed on the new pipeline setup.

These results demonstrate that the project successfully met its objectives: the data processing pipeline reliably ingests streaming data and the graph analytics components produce meaningful results on the ingested data.

4. Discussion

This project demonstrated how to build a scalable and fault-tolerant data processing system using modern tools such as Docker, Kafka, Kubernetes, and Neo4j.

Scalability and Architecture Benefits: Moving from a simple Docker-based setup in Phase 1 to a full Kubernetes-managed pipeline in Phase 2 significantly improved system scalability and flexibility. In the initial setup, Neo4j operated alone, which limited the system's capacity. In contrast, the Phase 2 architecture allowed components like Kafka brokers and Neo4j instances to be scaled independently as needed. Kubernetes simplified deployment, resource allocation, and management, making it easier to adapt the system for larger data volumes or cloud environments.

High Availability: Phase 2 also addressed reliability. By using Kafka as a buffer, the system ensured that incoming data would not be lost even if Neo4j was temporarily unavailable. Kubernetes further enhanced availability by monitoring and restarting failed components automatically. Together, these features created a robust pipeline that could handle service interruptions and maintain continuous operation.

Challenges Faced: Building the project involved solving several technical challenges. One of the early difficulties during Phase 1 was setting up Neo4j to run custom graph algorithms. Initially, attempts to use the Graph Data Science (GDS) procedures, such as checking the library version or running PageRank, resulted in errors. This happened because the GDS plugin was either not installed or not properly authorized. Following Neo4j's official documentation, the issue was fixed by manually installing the GDS plugin and updating the neo4j.conf file to allow unrestricted procedure calls. This step provided important insight into Neo4j's security model and how external plugins are added to expand its capabilities.

Another challenge came during Phase 2 while working with Kubernetes. Proper configuration in the Kubernetes YAML files and Helm charts was critical for deploying services like Kafka and Neo4j. Errors such as incorrect service names or missing environment variables often caused connection failures between components. Diagnosing these problems involved examining pod logs (kubectl logs) and

using resource descriptions (kubectl describe) to identify causes like CrashLoopBackOff errors or liveness probe failures. These issues were gradually resolved by adjusting configurations and systematically troubleshooting deployment problems.

Networking within the Kubernetes cluster also presented difficulties. For example, the Kafka-Connect sink needed to communicate with the Neo4j database through the correct internal address. In the configuration, the Neo4j service had to be specified as bolt://neo4j-service:7687. If there were mistakes in the hostname or port, connection attempts failed. Similarly, the data producer required the correct Kafka bootstrap server address based on the internal Kubernetes service setup. These problems highlighted the importance of ensuring that hostnames and ports were correctly aligned across all services to achieve reliable communication.

Reflection and Future Work: Completing this project created a strong foundation for future enhancements. One possible next step is to add real-time analytics or alerting capabilities. For instance, stream processing engines like Apache Spark or Kafka Streams could be attached to Kafka to analyze trip data before it reaches Neo4j. This could allow early detection of patterns, such as identifying congestion on busy routes in real-time.

Another improvement could involve dynamically updating graph algorithms. PageRank scores, for example, could be recalculated regularly as new trip data streams in, showing how the importance of different locations changes over time. This could be automated using scheduled jobs in Kubernetes or through Neo4j's internal triggers.

The architecture also allows for easy expansion. More Kafka topics and multiple Neo4j databases could be introduced to support scenarios like analyzing trips from different cities or operating in a multi-tenant system. This would further demonstrate the flexibility of the streaming pipeline.

Additional data sources could also be integrated to make the graph richer and more detailed. For example, combining taxi trips with public transportation data or road network maps could support more complex queries, such as finding the best multi-modal routes. New producers and sink connectors could be added to the existing pipeline without major redesigns, and the graph model could be updated to include features like time-based properties or driver-related information.

Finally, moving the pipeline to a cloud platform such as AWS or Google Cloud would allow testing at a much larger scale. Using managed Kubernetes services, the system could be expanded across multiple nodes with more computing resources. Setting up continuous integration and deployment (CI/CD) pipelines for automating updates to the infrastructure and application code would further improve reliability and efficiency.

5. Conclusion

This project successfully demonstrated how to build a distributed pipeline for real-time graph data processing and analysis. In Phase 1, a Neo4j graph database was set up using Docker to store NYC taxi trip data, and algorithms like PageRank and BFS were implemented to extract insights. Phase 2 expanded the system by introducing Kubernetes and Kafka, enabling the pipeline to scale, manage streaming data, and maintain high availability.

The final system achieved near real-time data ingestion and immediate analysis of graph data. It showed how technologies like Docker (for containerization), Kubernetes (for orchestration), Kafka (for streaming), and Neo4j (for graph storage and computation) could work together to create a flexible and reliable pipeline.

The approach combined algorithm design with system deployment, providing a complete view of scalable data engineering. The techniques used here have broader applications, including social network analysis, recommendation engines, and smart city data management.

References

[1] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab, Technical Report, 1999.

[2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade," ACM Queue, vol. 14, no. 1, 2016.

[4] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in Proceedings of the NetDB Conference, 2011.

[5] I. Robinson, J. Webber, and E. Eifrem, Graph Databases: New Opportunities for Connected Data, 2nd ed., O'Reilly Media, 2015.

[6] NYC Taxi and Limousine Commission, "TLC Trip Record Data," 2022. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[7] M. Rodriguez and J. Neubig, "Graph Representations for Real-World Data," Communications of the ACM, vol. 62, no. 10, pp. 91-99, Oct. 2019.

[8] Neo4j Graph Data Science Documentation, Neo4j, 2024. [Online]. Available: https://neo4j.com/docs/graph-data-science/current/

[9] Kafka Connect Documentation, Apache Software. [Online]. Available: https://kafka.apache.org/documentation/#connect